

The big picture

The schema splits cleanly into two layers. **Dimensions** describe the who/what/where of an experiment — they change rarely and are shared by everything downstream. **Facts** are the actual measurements — one table per assay domain, each shaped to match what that domain's import function already produces as a tidy long tibble.

DIMENSIONS

domains · experiments · subjects ·
samples · sample_digestions ·
assays

FACTS

facs_stains · facs_measurements ·
elisa_measurements ·
histo_measurements

Entity map

SCHEMA

Ten tables: six dimensions (blue accent) that rarely change, four fact tables (amber accent) that grow with every import.

domains

domain_id PK
domain_name UNIQUE

experiments

experiment_id PK
experiment_code UNIQUE
experiment_name, project
description

subjects

subject_id	PK
mouse_id	UNIQUE
experiment_id	FK
cage, strain, generation	
sex, treatment_group	
dob...end_date	DATE

samples

sample_id	PK
subject_id	FK
tissue	NOT NULL
collected_at	DATE
subject_id, tissue	UNIQUE

sample_digestions

sample_id	PK/FK
total_weight, facs_weight	
vol_total	

assays

assay_id	PK
assay_name	
domain_id	FK
assay_name, domain_id	UNIQUE

facs_stains

facs_stain_id	PK
sample_id, assay_id	FK
vol_stained, count_method	
vol_resuspended, vol_measured	VOLUMETRIC
bead_vol, bead_conc, bead_path	BEAD
sample_id, assay_id	UNIQUE

facs_measurements

facs_measurement_id	PK
facs_stain_id	FK
population_full_path, population	
metric, value	
source_file, imported_at	
stain_id, path, metric	UNIQUE

elisa_measurements

elisa_measurement_id	PK
sample_id, assay_id	FK
cytokine, sample_id_raw, replicate	
value, unit, result_status	
source_file, imported_at	

histo_measurements

histo_measurement_id	PK
sample_id	FK
assay_id	FK, NULL
metric, value	
source_file, imported_at	

FIG. 1 — RELATIONSHIPS

domains → assays	classifies
experiments → subjects	enrolls
subjects → samples	yields
samples → sample_digestions	1:1, FACS only
samples → facs_stains	stained per panel
assays → facs_stains	protocol
facs_stains → facs_measurements	yields
samples → elisa_measurements	measured
assays → elisa_measurements	protocol
samples → histo_measurements	scored
assays → histo_measurements	protocol

From raw file to analysis WORKFLOW

Every domain follows the same pipeline: a raw export is parsed by an existing package function into a tidy long tibble, written into the matching fact table behind a **UNIQUE** constraint (so re-running an import is idempotent, not additive), then queried back out across experiments through the same `dplyr::tbl()` interface already in use today.

FIG. 2 – PIPELINE

1. Raw exports

.wsp FlowJo workspace · ELISA results workbook (per cytokine) · meta workbook



2. Package import functions

```
facs_import_wsp() · elisa_read_results() · meta_read() / meta_clean()
```



3. Tidy long tibbles, in-session

```
FileName x Population x metric · sample_id x cytokine x replicate · mouse_ID  
x tissue meta + volumes
```



4. DuckDB, this schema

```
dimensions: experiments / subjects / samples / assays · facts:  
facs_measurements / elisa_measurements / histo_measurements
```



5. Cross-experiment analysis

```
dplyr::tbl(con, ...) |> filter() |> collect() · analysis_*() / plot_*() /  
report_knit_*()
```

Why this shape

idempotent imports	Every fact table's UNIQUE constraint means re-running an import script can't silently duplicate rows — DuckDB rejects the collision instead of the script relying on a manual check-then-append.
facs_stains split	Volumes that vary per stained panel (vol_stained , vol_resuspended / vol_measured or the bead-counting equivalents) live one level below the sample, since one digested tissue sample is routinely split into several differently-stained tubes.
count_method branch	facs_stains carries both volumetric (HTS) and bead-based counting fields, with a CHECK constraint enforcing only the fields matching the chosen method are filled in.

domains lookup

Assay domains are rows in a table, not a hard-coded list — adding scRNA-seq or any future modality later is an `INSERT`, not a schema migration.

provenance

Every measurement row carries `source_file` back to the raw export it came from — the database never loses the thread to the original data.

Out of scope here: the R-facing `db_` functions that will read/write this schema, and migrating the existing `MPOmRNA.db` data into it, are separate design work that follows once this structure is locked in.